

SpecProto: A Parallelizing Compiler for Speculative Decoding of Large Protocol Buffers Data

Zhijie Wang
University of California, Riverside
Riverside, CA, USA
zwang711@ucr.edu

Chales Hong
University of California, Riverside
Riverside, CA, USA
chong039@ucr.edu

Dhruv Parmar
University of California, Riverside
Riverside, CA, USA
dparm003@ucr.edu

Shengbo Ma
University of California, Riverside
Riverside, CA, USA
sma081@ucr.edu

Zhijia Zhao
University of California, Riverside
Riverside, CA, USA
zhijia@cs.ucr.edu

Qidong Zhao
Google
Sunnyvale, CA, USA
qidongzhao@google.com

Xu Liu
Google
Sunnyvale, CA, USA
xliuprof@google.com

Abstract

Protobuf is a widely used data serialization format, especially in cloud environments. However, existing compilers generate only serial decoders, limiting scalability for large datasets. While parallel parsing has been studied for textual formats (e.g., XML), parallel decoding of binary formats like Protobuf remains unexplored, which present unique opportunities.

We propose two techniques to enable parallel Protobuf decoding. First, we leverage the length-prefixed encoding of Protobuf to "skim" the binary and identify decoding tasks. To address inefficiencies caused by many small or imbalanced fields, we further introduce speculative parallelization, which partitions a binary into even segments and predicts decoding states across boundaries. We implement these techniques in SpecProto, a parallelizing compiler that generates parallel decoders for a given Protobuf schema. Experiments on real-world and synthetic datasets show that SpecProto achieves significant speedups by leveraging multiple CPU cores.

CCS Concepts: • **Computing methodologies** → **Parallel algorithms**; • **Computer systems organization** → **Multicore architectures**.

Keywords: Protocol Buffers, Parallel Decoding, Binary Data Processing, Multicore Systems, Speculative Parallelization

ACM Reference Format:
Zhijie Wang, Chales Hong, Dhruv Parmar, Shengbo Ma, Zhijia Zhao, Qidong Zhao, and Xu Liu. 2026. SpecProto: A Parallelizing Compiler for Speculative Decoding of Large Protocol Buffers Data. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '26), March 22–26, 2026, Pittsburgh, PA, USA.* ACM, New York, NY, USA, 15 pages. https://doi.org/10.1145/3779212.3790225



This work is licensed under a Creative Commons Attribution 4.0 International License.

ASPLOS '26, Pittsburgh, PA, USA
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2359-9/2026/03
https://doi.org/10.1145/3779212.3790225

1 Introduction

Protocol Buffers (Protobuf) [7] is a language-neutral and platform-neutral binary data format for serializing structured data. It has been widely adopted in cloud environments for inter-server communications [12] and data storage, such as Google's Dremel/BigQuery [20], and serves as the foundation of gRPC, a leading standard for remote procedure calls.

Binary vs. Textural Serialization. Unlike textual data formats, such as JSON and XML, that embed attribute names in the serialized data (e.g., `"name":"Alex"`), Protobuf avoids this overhead by enforcing a predefined schema—*messages*, which are stored in a `.proto` file. As illustrated in Figure 1, given a `.proto` file, the Protobuf compiler generates a pair of encoder and decoder in a specified language, for data serialization and deserialization. Study shows that Protobuf messages can be up to 10× smaller than equivalent JSON messages [28] and can perform up to 6× faster than JSON in Java-to-Java server communications [15].

Besides Protobuf, some other binary serialization formats include Apache Thrift [29], MessagePack [21], BSON [23], and Apache Avro [3]. While this work focuses on Protobuf, some of the issues and ideas could apply to other binary formats.

Scalability Concern in Protobuf Decoding. The decoding efficiency of Protobuf has become increasingly critical amid surging data transmission and storage demands. However, the state-of-the-art Protobuf compilers, including the one from Google, only generate sequential decoders, limiting

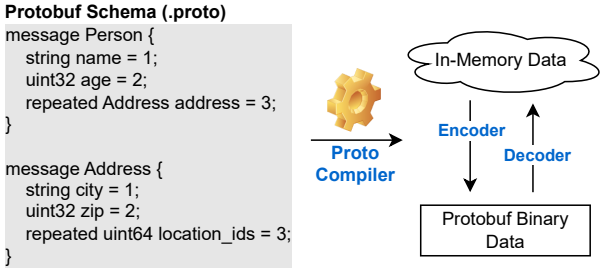


Figure 1. Protobuf Workflow.

their scalability on multicore CPUs, resulting in performance bottlenecks for large datasets. For example, Go pprof [30] stores profile data in Protobuf reaching sizes of gigabytes. Without the ability to harness multiple cores, processing such large datasets faces significant performance limitations, which has drawn complaints from developers [31].

Parallelization Challenges. The main reason Protobuf has traditionally adhered to a serial decoding design is the inherent dependencies in data encoding. When encoding an in-memory object, its fields are packed sequentially into a contiguous memory block. Figure 2 shows the encoding of a `Person` object according to the schema in Figure 1. Due to the presence of variable-length fields (e.g., varint, string, or submessages), determining the starting byte of any given field (except for the first one) appears impossible without decoding the entire message sequentially.

One may modify the encoding strategy to facilitate parallel decoding [4]. However, this change comes with a significant drawback: the loss of compatibility, which undermines the viability of Protobuf for inter-server communication.

Prior studies have explored the parallelization of textual data processing, including XML parsing and querying [10, 19, 24] and JSONPath querying [9]. While these studies offer valuable insights, Protobuf, as a binary serialization format, presents distinct opportunities and challenges.

Specifically, existing techniques for parallelizing textual data rely on explicit structural characters (e.g., braces, brackets, or quotes) to recover parsing context or guide speculation [8, 9, 11]. Such characters are absent in Protobuf's binary format. Instead, Protobuf decoding must recover both field boundaries and field types directly from raw bytes. Moreover, due to Protobuf's encoding scheme, the same binary tag may correspond to multiple field types across different messages, introducing type ambiguities that are unique to schema-enforced binary formats. As a result, prior approaches for textual data cannot be directly applied.

1.1 Contributions of This Work

In this work, we leverage *two key properties* of Protobuf—shared by other binary serialization formats—to parallelize its decoding. To our best knowledge, this is the first work on parallelizing binary deserialization while preserving its core compatibility. Next, we present the two key properties and their enabled parallelization techniques.

P1: Length-Prefixed Encoding. Unlike textual formats, Protobuf encodes the length information to delimit fields with variable lengths, such as strings, bytes, or embedded messages. As shown in Figure 2, a length-delimited field comprises three parts: *tag*, *length*, and *value*. A tag defines the field's wire type while the length indicates the number of bytes the field takes. For example, the field `name`'s tag is followed by "04", which means the next four bytes "41 6C 65 78" are the field value—the ASCII codes for string "Alex".

Skimming-based Parallelization. Our first attempt is to leverage the encoded lengths, which makes it possible to "skip" a field value without examining its bytes and bits in detail (impossible for JSON or XML). This feature allows us to quickly skim the Protobuf binary to identify the top-level fields without decoding them, such as `name`, `age` and `address` in Figure 2. After that, the extracted fields are decoded in parallel. With proper pre-memory allocation, their results can be automatically merged. While reasonable in principle, this approach has two significant drawbacks.

- *Serial bottleneck.* The first phase—skimming, remains serial and could become a performance bottleneck when the message contains repeated fields—similar to an array, it may include many field instances.
- *Load imbalance.* When the message contains only a few fields or if their sizes vary significantly, decoding them in parallel may suffer from load imbalance.

To address these limitations, we propose an alternative parallelization strategy that removes the need of skimming. by taking advantage of another property of Protobuf.

P2: Schema-Enforced Encoding. Unlike JSON and XML, which are known to be "schemaless", Protobuf (and some other binary formats) strictly follows a schema for encoding and decoding. As illustrated in Figure 1, the schema defines the structures of messages, specifying the type and number (ID) of each field within the message and its submessages.

Speculative Parallelization. Our second attempt aims to speculatively start decoding at "intermediate points" within the Protobuf binary. This may seem extremely challenging, given that all information is encoded as raw bits (0s and 1s). However, by leveraging the Protobuf schema specification, we find it is possible to speculate the decoding state with high accuracy, thus enabling effective speculative parallelism.

We break down the speculative decoding problem into two subproblems: (1) *tag speculation* and (2) *type speculation*.

Tag Speculation. Given an arbitrary "intermediate point" in the binary, the speculative decoder first locates a tag as its starting point for decoding. Not every byte could be part of a tag. Only bytes or byte sequences encoded according to

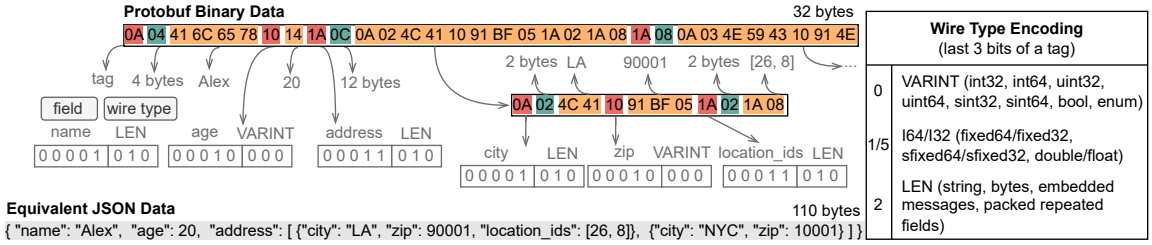


Figure 2. Example Protobuf message in hexadecimal format. (Its schema is defined in Figure 1; an equivalent JSON string is shown at the bottom; Each field is encoded using a tag: field name and its wire type; For the LEN-type field, its length in bytes is also encoded.)

the schema (the `.proto` file) are valid. Based on this insight, we derive an automaton that can automatically search for valid tags. However, a significant challenge remains—*fake tags*—bytes that coincidentally align with valid tags. We will address this together with the type speculation.

Type Speculation. After locating a tag, the next step is decoding the field value according to its type. However, the mapping between fields and field numbers is not always unique. In fact, it is common for submessages to reuse field numbers of their ancestors, leading to *type ambiguity*. To address this ambiguity as well as the fake tags, we compile the schema into an *type transition automaton (TTA)*, which captures the *following relations* between the types of different fields. During speculative decoding, we leverage the TTA to infer the feasible types. To ensure the correctness, after parallel speculative decoding, we include a validation and merging phase. In cases of misspeculation, the corresponding binary chunks get (partially) re-decoded.

With the above speculation techniques, we can simply partition the Protobuf binary into equal-size chunks, process them in parallel, and merge the results.

Evaluation. We prototyped a Protobuf compiler, SpecProto, that integrates both parallelization techniques. We evaluated its performance on datasets from real-world applications and synthetic schemas. Results show that skimming-based and speculative parallelization yield highly efficient parallel decoders, achieving average speedups of 3.8× and 4.9× over serial decoding on a 16-core CPU, respectively.

2 Background

In this section, we provide more background information on data serialization and Protocol Buffers.

2.1 Data Serialization and Deserialization

When in-memory objects are transmitted over a network or stored in a file, they are first serialized into a byte sequence. Later, when needed, they are retrieved and reconstructed into their in-memory representation—deserialization [14]. It is common to utilize built-in libraries from programming languages, such as `java.io.Serializable` for such purposes. However, these libraries are language-specific, thus posing challenges in multi-language development environments. To meet this need, language-agnostic serialization formats—both textual and binary—have been proposed.

Textual data formats, such as JSON, are widely used for their human readability, which facilitates debugging and manual editing. However, due to the lack of a strict schema, each attribute in these textual formats must carry its name information, like `"city":"LA"` in JSON and `<city>LA</city>` in XML, which not only increases the size of serialized data, but also slows down data decoding.

Binary data formats, like Protobuf, address the limitations of textual data formats by requiring predefined schemas. This schema-based approach eliminates the need of field names, resulting in more compact data representations and improved decoding speed. Binary formats are particularly suited for applications where data size and decoding time are critical, such as high-performance microservices, real-time systems, and data-intensive distributed applications.

2.2 Protocol Buffer

Protobuf was originally developed by Google and became its most widely used data format [7]. Figure 1 illustrates its basic workflow. Users begin by defining a schema (i.e., *messages*) in a `.proto` file. This schema is then compiled using the Proto compiler, which produces an encoder and a decoder in a specified language. For C++, the compiler generates a class for each defined message, providing simple accessors for its fields and methods to serialize the structure into a byte sequence and deserialize it back into its original form.

Figure 2 illustrates a `Person` message with an embedded `Address` message. A unique field number is assigned to each field of a message for encoding the field. The field type and number are encoded into a VARINT, called a *tag*, based on the formula `(field_number << 3)|wire_type`, where the `wire_type` (see the table in Figure 2) dictates how to decode the subsequent value. For example, when `wire_type` = 0, the following value is of VARINT type; when `wire_type` = 2, the next value has a variable length and the length is encoded first followed by the bytes of its value. In addition, a field can be `repeated`—it may appear multiple times in the binary data (see `Address` in `Person` in Figure 2). A repeated field would be

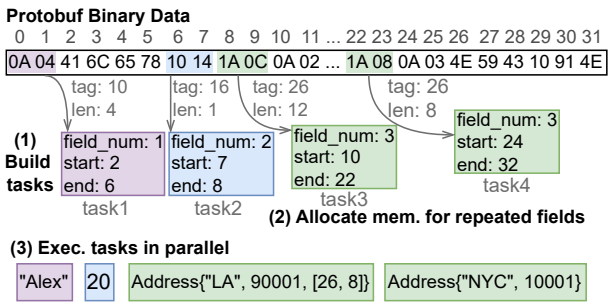


Figure 3. Illustration of Skimming-based Decoding.

decoded into a list. Note that fields do not obey any specific order during the encoding.

3 Skimming-based Parallelization

In this section, we present our first attempt to parallelize Protobuf decoding, by taking advantages of the encoded field lengths to quickly skim the binary message.

In practice, large Protobuf messages are often composed of many repeated LEN-type fields. As discussed in Section 1, LEN-type fields use length-prefix encoding. Based on this, we propose a two-pass parallelization approach. In the first pass, the binary data is quickly skimmed to locate the boundaries of LEN-type fields in the message without decoding any field values. In the second pass, the identified fields are decoded in parallel. Figure 3 illustrates this parallel decoding process, with more details explained in Algorithm 1.

Step 1: Skimming. The first pass is handled by the Skim() function. Given the binary data, it begins by reading the tag (Line 5) and then decodes it according to Protobuf's field encoding logic, resulting in the field number and wire type. The function then calculates the length of the field value based on the wire type (Lines 8-12). Next, it records the start and end positions of the field value, which defines a *decoding task*, and groups them by field number (Line 13). Then, it jumps over the field value based on its length and proceeds to the next tag, until it reaches the end of the binary.

Step 2: Memory Allocation. Once these decoding tasks (field values) are collected, memory must be allocated to store the results of the subsequent parallel decoding. To deal with the memory allocation for repeated fields, we group tasks by field such that the size of each field group corresponds to the final size of the decoded list, and the order of tasks within a field group determines their indices in the decoded list.

Step 3: Parallel Decoding. Finally, the whole collection of decoding tasks is processed by ParallelDecode(), which is a method of the `Person` class. ParallelDecode() sets the fields of the class by calling each field's corresponding (serial) decoding function, such as ReadString() and `address`.Parse(). This allows seamless integration of existing optimizations for

Algorithm 1: Skimming-based Parallel Decoding

```
1  function Skim(data):
2      Initialize tasks /* tasks grouped by field_num */
3      index ← 0
4      while index < length(data) do
5          tag ← ReadTag(data, index)
6          field_num ← tag ≫ 3 /* decode field number */
7          wire_type ← tag & 7 /* decode wire type */
8          switch wire_type : /* find field length */
9              case 0 : len ← ReadVarintLen(data, index)
10             case 1 : len ← 8
11             case 2 : len ← ReadLen(data, index)
12             case 5 : len ← 4
13         append (index, index + len) to tasks[field_num]
14         index ← index + len /* skip the field value */
15     return tasks
16 /* Generated for each message. Here is for Person. */
17 function ParallelDecode(data, tasks):
18     parallel for field_num, field_tasks in tasks do
19         switch field_num :
20             case 1 : /* decode the bytes into a name */
21                 name ←
                     ReadString(data, last(field_tasks))
22             case 2 : /* decode the bytes into an age */
23                 age ← ReadVarint(data, last(field_tasks))
24             case 3 : /* decode the bytes into addresses */
25                 address ← AllocMem(length(field_tasks))
26                 parallel for i, task in field_tasks do
27                     address[i] ← new Address()
28                     address[i].Parse(data, task)
```

sequential decoding, like the Bit Manipulation Instruction Set 2 (BMI2) to accelerate VARINT parsing [18].

Note that a similar skimming approach can also be applied to parsing textual data formats, like XML and JSON, however, the skimming overhead would be significantly higher, as textual data lacks length encoding. Thus, skimming must examine the text character by character, creating a serious issue that may outweigh the parallelization benefits [19].

Limitations. Two critical assumptions must hold to make skimming-based parallelization effective. First, the binary message should contain a moderate number of fields—too many fields can slow down skimming, exacerbating the serial bottleneck, while too few may not yield enough decoding tasks to fully utilize the CPU cores. Second, the field sizes should be relatively balanced to prevent any single task from disproportionately dominating the total parallel decoding time. To address the above limitations, we next introduce an alternative approach that offers more flexible parallelism.

4 Speculative Parallelization

Speculative parallelization has proven effective in processing textual data, such as matching regular expression [34] and

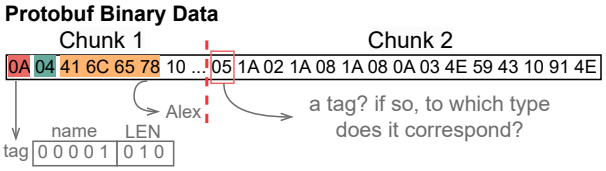
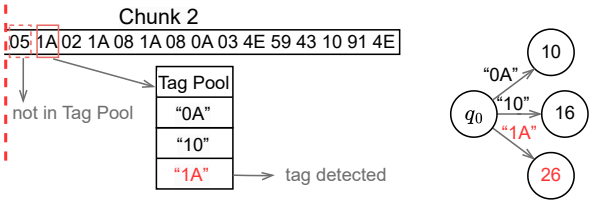


Figure 4. The Challenge of Speculative Parallelization.



(a) Tag Detection Process **(b)** Tag Pool DFA

Figure 5. Tag Speculation

XML/JSON querying [8, 9, 24]. However, speculating the decoding state from a raw bitstream is more challenging as it lacks structural characters (e.g., "{" or ":") that provide clear cues in human-readable textual data.

A Motivational Example. Figure 4 illustrates a binary message split into two chunks. The first chunk starts with byte "0A", which, by tag encoding, denotes a LEN-type field with field number 1. This corresponds to the top-level field `name`, providing a clear initial decoding state.

However, initial decoding state for the second chunk is less obvious: *is its first byte ("05") the start of a valid tag? If so, to which field does it correspond?* This ambiguity arises because preceding fields might have variable lengths, causing arbitrary partitioning boundaries to potentially fall in the middle of a field. Moreover, correctly speculating the first tag and type alone does not guarantee that the decoding state for subsequent fields will become clear. As decoding proceeds further into the chunk, similar ambiguities can repeatedly occur, requiring ongoing speculation.

In the following, we will demonstrate that it is possible to effectively speculate on the decoding state(s) by leveraging structural information embedded in the `.proto` file. For better illustration, we decouple the decoding-state speculation into *tag speculation* and *type speculation*.

4.1 Tag Speculation

Recall the question in decoding the second chunk in Figure 4: *Could "05" represent a valid tag?* In theory, any byte value between 0x01 and 0xFF could appear as part of a valid tag. However, given the schema defined in Figure 1, "05" cannot be part of a valid tag. To systematically answer this question, we need to find out what tags are possible for a given schema.

According to Protobuf encoding, each field has a unique *field number*, which, combined with a *wire type*, forms a binary-encoded identifier—a *tag*. Given a Protobuf schema, we can infer all of its valid tags, referred to as its *tag pool*.

Definition 4.1. Given a Protobuf schema $\mathcal{S}$, its *tag pool* $\mathcal{T}(\mathcal{S})$ refers to the set of all unique tags that may appear in the encoded message, as determined by the schema.

Considering the example in Figure 2, we have $\mathcal{T}(Person)$ = {"0A", "10", "1A"}. Note that in general a tag may consist of multiple bytes for larger Protobuf schemas.

A straightforward way to finding a valid tag is directly scanning the binary sequence using this tag pool starting from the partitioning position, as illustrated in Figure 5-(a). As tags may consist of multiple bytes, a more structured realization of this idea is compiling the tag pool into an automaton, as shown in Figure 5-(b).

4.1.1 Tag Pool DFA.

Deterministic finite automaton (DFA) is a classic way for identifying string patterns in textual data. In this work, we draw an analogy between binary tags and words in textual content. Specifically, we *treat each byte in an encoded Protobuf message as a character in an alphabet*, allowing us to interpret tags from the tag pool as the "words" to be matched within this byte stream. We refer to this DFA for matching tags in a tag pool as the *Tag Pool DFA*.

Definition 4.2. Given a Protobuf schema $\mathcal{S}$, its *Tag Pool DFA* is a five-tuple $D = (Q, \Sigma, \delta, q_0, F)$ where:

- Q is a finite set of states, representing the matching progress of valid tags defined in $\mathcal{T}(\mathcal{S})$.
- Σ is the input alphabet consisting of all possible values of individual bytes (0x01 to 0xFF).
- $\delta : Q \times \Sigma \to Q$ is the deterministic transition function mapping a state and an input byte to the next state.
- $q_0 \in Q$ is the initial state—no input yet matched.
- $F \subseteq Q$ is the set of accepting states, each indicating a complete match of a valid tag in $\mathcal{T}(\mathcal{S})$.

By transitioning through DFA states based on each input byte, valid tags can be efficiently detected. For example, the tag pool DFA in Figure 5-(b) can recognize "05" as an invalid tag, while correctly identifying the next byte "1A" as valid.

Algorithm 2 outlines the process of generating a tag pool DFA for a given Protobuf schema $\mathcal{S}$. The construction starts by first encoding each field's valid tag (determined by its field number and wire type) into VARINT format, and then inserts each encoded tag into the DFA.

4.1.2 Fake Tags and Tag Speculation.

It is critical to note that the tags identified by the tag pool DFA are valid tags but not necessarily actual tags because the field values may contain bytes that resemble valid tags.

Definition 4.3. In the process of tag identification, a byte or sequence of bytes that resembles a valid tag from the tag pool but is not an actual one in the binary is a *fake tag*.

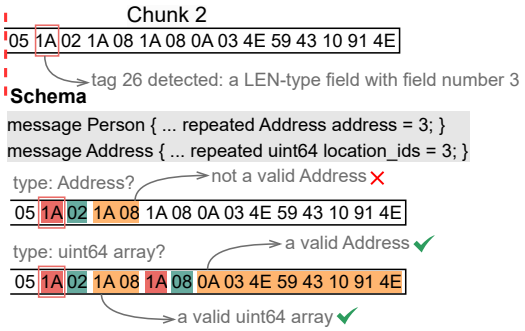


Figure 6. Illustration of Tag-Field Ambiguity

Because fake tags may exist, the tags identified by the tag pool DFA cannot be guaranteed to be correct. In fact, it is very challenging to tell whether an identified tag is fake or not due to the missing of full decoding context. To address this challenge, we first assume an identified tag is real (i.e., speculation) and defer the fake tag detection (i.e., validation) to later phases of the parallel decoding process.

Algorithm 2: Tag Pool DFA Generation

```
1  function BuildTagPoolDFA(S):
2      q0 ← new DFA state (initial state)
3      for field f ∈ S do
4          tag ←
             EncodeVarint(f.number, wireType(f.type))
5          current_state ← q0
6          for byte b ∈ tag do
7              if b ∉ current_state.transitions then
8                  current_state.transitions[b] ← new DFA
                     state
9              current_state ← current_state.transitions[b]
10         mark current_state as accepting state
11         current_state.tag ← tag /* map tag to a state */
12     return constructed DFA
```

After identifying a valid tag, we then move to the more complex problem—determining the field type.

4.2 Type Speculation

Given a tag, we can deterministically decode its field number and wire type based on Protobuf's tag encoding scheme. For example, tag "1A" decodes to field number 3 with a wire type of LEN. However, mapping this decoded information to specific message fields may be non-deterministic.

4.2.1 Tag-Field Ambiguity. Figure 6 depicts an example where tag "1A" can be decoded into either an `address` of a `Person` or the `location_ids` of an `Address`. Two key factors contribute to this phenomenon:

- *Field Number Reuse*: Field numbers are only unique within the scope of a single message and are frequently reused across different message types. In this case, `address` and `location_ids` both use field number 3.
- *Field Type vs. Wire Type*: Protobuf employs a coarse-grained wire type system that lacks the full richness of field types (e.g., both `repeated Address` and `repeated uint64` are mapped to the LEN wire type).

The first factor is the root cause of the ambiguity, while the second increases the likelihood of such ambiguity occurring in practice. Next, we define this ambiguity formally.

Definition 4.4. Schema $\mathcal{S}$ is *tag-field ambiguous* if there exist two fields f_1 and f_2 of two different messages that are encoded using the same tag.

We address tag-field ambiguity in two phases. First, we focus on a special case of it, called *tag-type ambiguity*, which is formally defined below. The remaining aspects of tag-field ambiguity are resolved during the validation phase.

Definition 4.5. Schema $\mathcal{S}$ is *tag-type ambiguous* if there exist two fields f_1 and f_2 in different messages that share the same tag but have different field types, i.e., $T(f_1) \neq T(f_2)$.

Obviously, a tag-type ambiguous schema must be tag-field ambiguous; but the converse does not necessarily hold.

Tag-type ambiguity complicates speculative decoding. As demonstrated in Figure 6, an attempt to interpret the tag as an `Address` fails; the correct type is a `uint64 array`. To reduce the likelihood of such failures, we next introduce a special automaton to help infer the field type associated with a given tag, called *type transition automaton*.

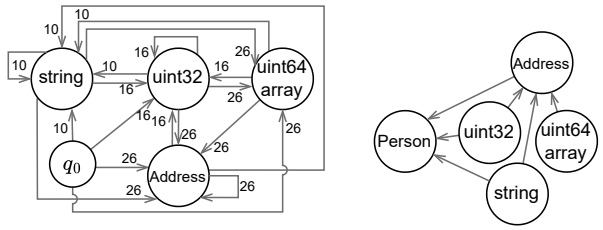
4.2.2 Type Transition Automaton (TTA). The intuition behind this automaton is to capture the "following relation" between types in the encoded data. For example, given that the last decoded field is an `Address`, the automaton is able to determine which field types could validly follow next. A formal definition is given below.

Definition 4.6. The *type transition automaton (TTA)* of a schema $\mathcal{S}$ is defined as a 4-tuple $M = (Q, \Sigma, \delta, q_0)$ where:

- Q is a finite set of type states. Each type state, except for q_0, represents a unique field type specified in $\mathcal{S}$.
- Σ is the input alphabet, consisting of all valid tags derived from the schema, that is, the tag pool $\mathcal{T}(\mathcal{S})$.
- $\delta : Q \times \Sigma \rightarrow 2^Q$ is the transition function, which maps a type and a tag to a set of possible successor types.
- $q_0 \in Q$ is the initial state, representing cases where no type information is yet known.

Example Usage. Consider the TTA in Figure 7-(a) and the type speculation scenario in Figure 6. If the previous type is unknown and the tag is "1A", this field could be `Address` or `uint64 array`. In contrast, if the previous field type is `uint64 array` and the tag is "1A', then this field must be `Address`.

Before discussing the TTA construction, we first introduce another relevant concept, called *type graph*.



(a) Type Transition Automaton **(b)** Type Graph

Figure 7. Example TTA and Type Graph (states and nodes in TTA and type graph are referred to as *type states* and *type nodes*.)

Definition 4.7. The *type graph* of a schema $\mathcal{S}$ is a directed graph where nodes represent message or field types in $\mathcal{S}$, and directed edges denote their "composition relation". The top-level message of $\mathcal{S}$ is called the *root* of the type graph.

Figure 7-(b) shows a type graph, where `Person` serves as the root. Since only message types can be composed of other types, *all edges of a type graph point to message-type nodes.*

Example Usage. Type graph can guide TTA construction. In Figure 7-(b), nodes `Address`, `int32`, and `string` all point to `Person`, which is the root. This implies that fields of type `int32` or `string` can directly follow an `Address`. On the other hand, if `Person` is not the root, say `Person` and `Pet` both point to `PetOwner`, then a `Pet` field could also follow an `Address`.

TTA Construction. Algorithm 3 describes the construction of TTA for a schema $\mathcal{S}$. It first builds the type graph, then iterates over fields in $\mathcal{S}$ to capture valid transitions.

For each field f in $\mathcal{S}$, the algorithm adds a transition from the initial state q_0 to its corresponding type state (Line 7), indicating that this is a possible type of the first field after a partition point. Next, it starts a fresh DFS traversal over the type graph (i.e., *outedges*) from the type node of field f (Lines 9–10). The DFS function takes two inputs: the *start* type node and the current type node u (Line 20). When the current node u becomes the root (i.e., `Person` in our example) or has been visited already, the DFS returns; otherwise, the algorithm marks u as visited (Line 23) and starts to visit the out-neighbors of the current type node. Then, for each field f' of an out-neighbor, the algorithm inserts an edge from *start* to f's type node in the TTA (Lines 26–29). An exception occurs when u is not a repeated submessage: in this case, a type node u does not immediately point to itself in the TTA (Lines 27–28). For example, in Figure 7-(a), `uint64 array` does not form a self-loop. However, a type node may still point to itself through traversal of other message types in DFS. Finally, the algorithm starts a recursive call to DFS with the same *start* node, but now the current node u becomes the out-neighbor (Line 30). The above process repeats until all fields of schema $\mathcal{S}$ have been processed in a similar recursive way. The final output would be a TTA.

Algorithm 3: TTA Construction

```
1  function BuildTTA(S):
2      q0 ← new NFA state (initial state)
3      initialize TTA /* TTA[ST][tag] stores the list of
         successor states for each state ST via a given tag */
4      outedges ← BuildTypeGraph(S)
5      for field f ∈ S do
6          /* add a transition from q0 to its type */
7          add f.type to TTA[q0][f.tag]
8          /* add derived transitions recursively via DFS */
9          reset visited to empty
10         DFS(f.type, f.type)
11     return TTA
12 /* Construct the type graph for guiding TTA construction */
13 function BuildTypeGraph(S):
14     initialize edges /* edges[T] lists out-neighbors of type T
         */
15     for message m ∈ S do
16         for field f ∈ m.fields do
17             add m.name to edges[f.type]
18     return edges
19 /* DFS on type graph to insert transitions into the TTA */
20 function DFS(start, u):
21     if u is root ∨ u ∈ visited then
22         return
23     add u to visited
24     for type T ∈ outedges[u] do
25         /* T is a message type; T.fields store its fields */
26         for field f' ∈ T.fields do
27             if f'.type = u ∧ u = start ∧ u is not repeated
                 submessage then
28                 continue
29             add f'.type to TTA[start][f'.tag]
30         DFS(start, T)
```

The reason edges are recursively added according to the type graph is to accommodate the hierarchical structure of fields in the schema. For instance, an `Address` might directly follow an `uint64 array` because `uint64 array` may represent the end of a preceding `Address`, and thus an edge needs to be added between the two type nodes.

With the above type inference technique, we next explain the type speculation process, referred to as *backtracking*.

4.2.3 Backtracking.

TTA is a non-deterministic—it may yield multiple possible field types. This uncertainty arises at each decoding step. To manage the accumulated ambiguity across fields, we record the candidate types of previous fields and backtrack to explore alternative possibilities if decoding fails after all options are exhausted. Additionally, we cannot avoid backtracking by converting NFA to DFA because, in an equivalent DFA, a state may correspond to multiple types (e.g., `uint64` and `Address`). As a result, we still need to try

Algorithm 4: Backtracking

```
1  /* Initially, index = DetectTag() and prior_type = q0 */
2  function SpecDecode(chunk, index, prior_type, results,
     TTA):
3      tag ← ReadTag(chunk, index)
4      possible_types ← TTA.next(prior_type, tag)
5      for type ∈ possible_types do
6          result ← ParseField(type, chunk, index)
7          if result is invalid then /* speculative decoding
             failed */
8              continue
9          append result to results
10         i ← result.next_index
11         if i exceeds chunk's size then
12             return success
13         /* speculatively decode the next field */
14         if SpecDecode(chunk, i, type, results, TTA) succeeds
             then
15             return success
16         else
17             remove the last item from results
18     return failure
```

each of these types one by one and track which types have not yet been tried.

Algorithm 4 describes the backtracking process. First, the inputs to the decoding function include a binary chunk, a starting index, the type of the previous field, a data structure for storing decoding results, and the TTA. It first extracts a tag, then determines the possible types based on the prior type and the tag using TTA (Lines 3–4). It then iterates over these possible types, attempting to decode the field for each one. If decoding fails (e.g., type mismatch), the algorithm continues with the next candidate (Lines 7–8); otherwise, it appends the result to *results* and updates i to the next index in the chunk (Lines 9-10). If i exceeds the chunk size, the function returns success, indicating the entire chunk has been decoded; otherwise, it recursively processes the next field with updated parameters. If the recursive call succeeds, success is propagated upward; otherwise, the last inserted result is removed and the function backtracks to explore other candidates (Lines 14–17).

Initially, the starting index is set to the index of the first byte of a tag, detected by a tag pool DFA, and the prior type is set to the initial state q_0 of TTA.

Fake Tag Detection. After exhausting all possible types via backtracking, if decoding still fails, then the chosen start tag is *fake* (assuming no data corruption). Figure 8 shows an example where the byte "1A" is speculatively treated as a tag. The algorithm first tries decoding it as an `Address`, which fails, then as a `uint64 array`, which seems successful. However, the next byte "91" is invalid as a tag, revealing the

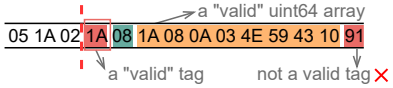


Figure 8. Fake Tag Detection

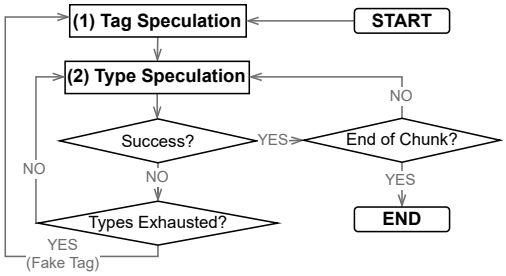


Figure 9. Workflow of Speculative Decoding

previous interpretation was wrong. With no other candidate types for "1A", the algorithm identifies "1A" as a fake tag.

When a fake tag is detected, tag speculation is restarted from the second byte of the fake tag or from the following byte if the tag is a single byte. Figure 9 illustrates the complete speculative decoding workflow over an input chunk.

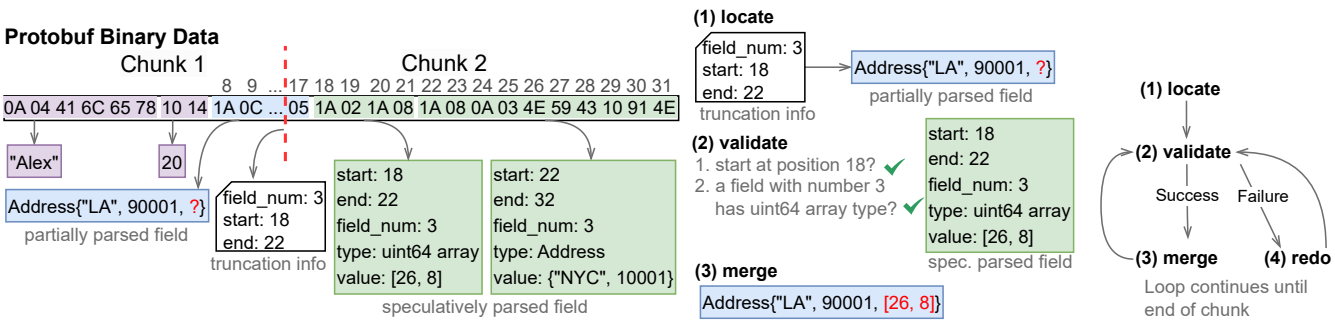
4.2.4 Optimizations.

We next discuss some opportunities to boost the effectiveness of speculative decoding in practice.

O1: Type Prioritization. During type speculation phase, if multiple candidate types are possible, we prioritize them based on the following two heuristics:

- **Prioritizing Submessage Types:** Compared to more generic types (such as string or uint64), submessages impose stricter constraints on byte sequence structure. Selecting them increases the likelihood of detecting misspeculation earlier if the choice is incorrect.
- **Prioritizing Self-Transitions:** If the last recognized field is a submessage T and T is among the candidate types, we prioritize T. This is based on the observation that repeated submessage fields often dominate the data and occur contiguously in the input. Favoring the same state improves speculation accuracy.

O2: Maximum-Cost Constraint. When a chunk begins in the middle of a large string field or a field containing a long sequence of raw bytes, the backtracking algorithm could spend a large portion of its time speculating tags and trying various candidate types. The speculation cost can accumulate quickly, degrading the overall performance. To address this, we impose an upper bound on the accumulated number of bytes accessed during speculative decoding for each chunk (e.g., 1.1× of the chunk size). Processing of the remaining chunk bytes will be deferred until the merge phase.



(a) Data Structures **(b)** Merge Process **(c)** Merge Workflow

Figure 10. Validation-Driven Merge

4.3 Validation-Driven Merge

After independent parsing of chunks, speculative results need to be validated and merged to form the final parsing result. Figure 10 illustrates this process.

Data Structures. First, we record speculatively parsed fields as well as the necessary parsing details for merging:

- **Speculatively Parsed Fields**. For each speculatively parsed field, we record its *start* and *end* positions, field number, inferred *type*, and parsed *value*.
- **Truncation Information:** When a field is parsed partially due to truncation, we call it a *tail field*. In this case, we record its field number and the range of the to-be-decoded part: (*start*, *end*), which can be calculated based on the decoded field length.

Figure 10-(a) shows two speculatively parsed fields for Chunk 2: a `uint64 array` and an `Address`, and the truncation information for Chunk 1. In particular, Byte 8 ("1A") indicates that the tail field has a field number 3 and Byte 9 ("0C") shows that it consists of 12 bytes, ranging from Bytes 10–21. The thread responsible for the first chunk actually parses up to Byte 17, because Bytes 15–17 correspond to a VARINT, and parsing stops only after this VARINT is parsed.

Note that when nested fields are truncated, we need to record multiple truncation information entries.

Merge Algorithm. Given two consecutive chunks i and $i+1$, and assuming chunk i has already been merged with the preceding chunks, the basic merge process consists of: (i) *locate*, (ii) *validate*, and (iii) either *merge* or *redo*.

- **Locate** the tail field F_{tail} of Chunk i, such as the field `Address`, truncated at Byte 18 in Figure 10-(b).
- **Validate** F_{tail}'s truncation information against the first speculatively parse field $F_{spec}[0]$ of Chunk $i+1$. It succeeds only if the following conditions are met.

$F_{tail}.start = F_{spec}[0].start \wedge$
$(F_{tail}.type = F_{spec}[0].type \;||\; \exists$ field $f \in F_{tail}.type$ s.t.
$f.type = F_{spec}[0].type$ when $F_{tail}.type$ is a message)

In Figure 10-(b), both fields start at Byte 18 and `Address` has a `uint64 array` field, therefore, $F_{spec}[0]$ is valid.

- **Merge or Redo:** If validation succeeds, the value of speculatively parse field ($F_{spec}[0]$) is merged into the results of Chunk i; otherwise, the algorithm re-parses from the failure point (e.g., Byte 18 in Figure 10-(b)).

If Chunk $i+1$ has more speculative parsed fields, repeat the validate–merge/redo steps, until end of the chunk.

When nested truncations occur, the algorithm processes truncation entries recursively from the leaf nodes upward, ensuring correct hierarchical merging. Algorithm 5 presents the detailed pseudocode for the merge algorithm.

Partial Redo. In practice, we find that usually only a small portion of data is incorrectly parsed, and these parsing errors typically do not affect other fields. To avoid re-parsing the entire chunk, we only re-parse the binary as necessary—until a successful decoding of a field.

5 Implementation

Based on our proposed techniques, we prototyped a proto compiler in Python, called SpecProto. SpecProto generates a parallel decoder in C++ for a standard Protobuf schema. Given a Protobuf binary data file, the decoder returns an in-memory object corresponding to the original message. It supports two parallelization modes: the speculative mode (see Section 4) and the non-speculative mode which uses the skimming-based parallelization (see Section 3).

SpecProto uses Jinja [26] as its templating engine for code generation. Given a schema and a parallelization mode, it produces a header and an implementation file containing the schema-specific parallel decoding logic. Each message in the schema corresponds to a C++ class. Scalar fields are stored using C++ primitive types, string fields are stored as `std::string`, repeated fields are stored in `std::vector`, and submessages are represented as pointers to instances of their respective types. In practice, users may modify the templates used in SpecProto to adopt custom data structures to accommodate their specific use cases.

Algorithm 5: Merge Algorithm with Partial Redo

```
1  /* Generated for each message. Here is for Person. */
2  function Merge(chunk, index, end, truncs, specs):
3      if truncs is not empty then
4          trunc ← pop first from truncs /* locate */
5          switch trunc.field_num :
6              case ... :
7              case 3 : /* the last address is partially parsed */
8                  address ← the last address
9                  address.Merge(chunk,
                           trunc.start, trunc.end, truncs, specs)
10         if the current message is still partially parsed then
11             AddNewTrunc(truncs, ...)/* for the next merge */
12             return
13     while index < end do
14         spec ← first spec from specs
15         if spec exists ∧ index = spec.start ∧
             TypeCheck(spec.field_num, spec.type) then /*
             validate */
16             switch spec.field_num :
17                 case ... :
18                 case 3 :
19                     append spec.value to address /* merge */
20             index ← spec.end
21             pop first from specs
22         else /* redo */
23             ParseOnce(chunk, index, end)/* parse one field
                 and write to the current Person message */
24             if the new field is partially parsed then
25                 AddNewTrunc(truncs, ...)
26                 return
27             PopInvalidSpec(specs)/* discard wrong specs */
```

When speculation mode is enabled, SpecProto generates the tag-pool DFA and TTA, stored in the implementation file, along with the speculative decoding workflow. The DFA and TTA are shared among worker threads. For the root message, a corresponding speculative decoding method is generated, which executes the three steps described earlier: partitioning, speculative decoding, and merging with validation. Each message implements a merge method to combine the results of speculative decoding. This is necessary because when a submessage is partially truncated, its merge method must be invoked to complete the partial decoding. For all messages, a sequential parsing method is generated. In particular, the first thread uses the sequential parsing method for the root message to parse a partial message. All threads except for the last one may use the sequential parsing method to parse a partial message and cause the truncation point.

In non-speculative mode, a serial parser is generated for each message, except for the root message, which employs the two-pass parallelization approach.

For parallel execution, SpecProto leverages OpenMP [25], which exhibits similar performance to an implementation using `std::thread` in our experimental setup. SpecProto uses `mmap` to map the input binary file into memory for efficient data access and mimalloc [22] for high-performance memory allocation.

6 Evaluation

In this section, we evaluate the performance of the parallel decoders generated by SpecProto. Note that their correctness has also been thoroughly tested and verified.

6.1 Methodology

We evaluate both the speculative and non-speculative modes of SpecProto and compare their performance against serial decoding, which serves as our baseline. For reference, our baseline runs about 2–4× faster than the standard `protoc` compiler (Protocol Buffers v23.4, `proto3`) [7], which offers more features, including arena allocation for memory management, unknown fields preservation, advanced language features (e.g., map and oneof), UTF-8 validation, and has a significantly larger codebase. We believe that most of these features are orthogonal to our parallelization techniques. One exception is unknown fields preservation in the speculative mode, since the speculative mode examines valids tags during the decoding phase.

For the schemas under our evaluation, the encoders and decoders (which share code) generated by `protoc` have an average size of 13442 lines. In comparison, our sequential decoder has an average size of 1574 lines, and our parallel decoders have average sizes of 6503 lines (spec) and 2558 lines (non-spec).

Table 1. Dataset Statistics (#msg and #field represent the number of messages and fields defined in the schema; R_{tag} is the tag density: tagBytes/totalBytes; d is the message depth)

Data	Size	R_{tag}	#msg	#field	#tl_field	d
PROF	1084MB	5.1%	8	45	14945918	3
MAP	733MB	4.0%	11	35	4439	6
PRD	846MB	1.4%	4	61	272503	3
TT	873MB	5.9%	8	82	2	5
SYN1	990MB	0.2%	6	29	2018	6
SYN2	1018MB	8.6%	6	29	3939	55
SYN3	851MB	2.5%	61	190	320000	8

Datasets. Table 1 lists the datasets used for evaluation. The first one is a native dataset of profiling data (PROF) collected from the Go profiler `pprof` [30]. The next three datasets converted the corresponding JSON datasets, including tweets (TT) [32], map directions (MAP) [5], and Walmart products (PRD) [33]. In addition, we synthesized three datasets to offer a wider range of data characteristics.

Table 2. Overall Decoding Time and Memory Overhead.

	End-to-End Time (s)			Memory Overhead (GB)		
	Baseline (serial)	SpecProto non-spec	SpecProto spec	Baseline (serial)	SpecProto non-spec	SpecProto spec
PROF	4.93	2.31	1.33	5.87	7.06	6.58
MAP	0.84	0.17	0.16	1.89	1.92	2.02
PRD	0.71	0.38	0.15	2.05	2.11	2.10
TT	1.50	1.52	0.32	2.74	2.74	2.86
SYN1	0.87	0.13	0.22	2.42	2.47	2.49
SYN2	2.21	0.38	0.39	3.40	3.42	3.56
SYN3	1.99	0.52	0.32	3.85	3.90	3.93

Experiment Environment. Experiments were run on a server node with two Intel Xeon E5-2683 v4 processors and 512 GB RAM. Unless otherwise specified, all parallel executions use a single socket with 16 threads. This server runs Rocky 8 and uses G++ 9.2.1. All programs are compiled with `-O3`. The timing results reported are the average of five runs.

6.2 Overall Performance

Execution time. Table 2 reports the end-to-end decoding time of evaluated methods while Figure 11 visualizes the speedups relative to the baseline. The results show that both `non-spec` and `spec` modes of SpecProto exhibit significant speedups over the baseline on almost all datasets, with the only exception being `non-spec` on the TT dataset.

More specifically, SpecProto `non-spec` achieves speedups ranging from 1× to 6.7× (3.8× on average), while SpecProto `spec` achieves speedups from 3.7× to 6.2× (4.9× on average). We will study the causes of speedup variations in Section 6.3.

Memory Overhead. The memory cost is important when parsing a single large message. Table 2 presents the memory footprints of different methods in the parsing process, which mainly consists of two parts: the binary data and the parsed results. Due to the presence of the VARINT type, the memory occupied by the parsed results is often greater than that of the original binary data. For most of the datasets, there is no significant difference in memory cost among the different methods when using 1 to 16 threads. For the PROF dataset, SpecProto consumes more memory than the baseline, especially for `non-spec`. This is because a very large number of objects are allocated simultaneously by all threads, which increases memory retained in the allocator's per-thread caches and leads to a higher peak memory overhead.

6.3 Impacts of Input Structure

Tag Density. For tag-dense datasets (R_{tag}>1%), `spec` often performs better than `non-spec`, because `spec` only requires a single scan—better locality, and its threads are assigned with almost balance loads. However, for tag-sparse datasets (e.g., SYN1), `non-spec` wins because the lower tag density makes `spec` less effective at speculating tags.

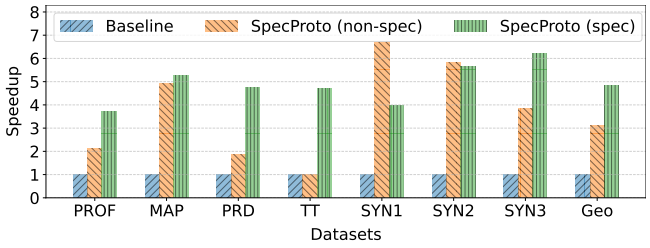


Figure 11. Speedups Compared to the Baseline.

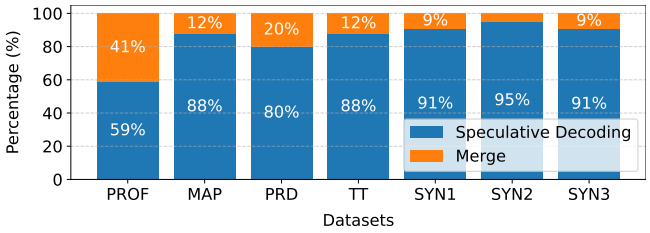


Figure 12. Time Breakdown (SpecProto-spec).

Top-Level Fields. In MAP and SYN2 datasets, the number of top-level fields is limited and their sizes are similar, in which case `spec` and `non-spec` perform similarly. When there are a large number of top-level fields (PROF, PRD, and SYN3), `non-spec` quickly loses due to the serial bottleneck in the first pass. Additionally, `non-spec` cannot achieve any speedup on the TT dataset because this dataset has only one large field at the top level, which means that during the parallel decoding phase, only one thread can be used.

Based on the results of the experiments on two specially synthesized datasets, SYN2 (d = 55) and SYN3 (#msg = 61), we did not observe that the deep nesting depth or large number of message definitions posed any unique challenges for the two parallelization algorithms.

6.4 Speculation Cost and Benefit Breakdown

Table 3. Speculation Cost Breakdown (in Bytes)

Data	Input	Processed (Ratio)	Redo
PROF	1,084,030,512	1,093,203,293 (101%)	9
MAP	732,708,874	732,715,106 (100%)	0
PRD	846,331,241	846,347,028 (100%)	0
TT	872,839,280	872,843,607 (100%)	160
SYN1	989,559,705	1,002,971,341 (101%)	86
SYN2	1,018,213,954	1,018,214,449 (100%)	101
SYN3	851,184,808	854,376,731 (100%)	8525

Figure 12 shows the breakdown of time spent on speculative decoding versus merging. For all datasets except PROF, the merging cost is negligible thanks to the lightweight redos (presented shortly). In the case of PROF, the merge process is more expensive due to the presence of a large object list (size

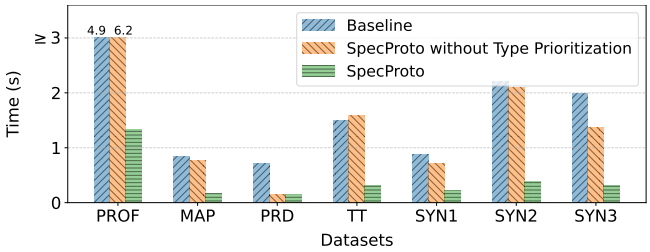


Figure 13. Benefits of Type Prioritization (SpecProto-spec).

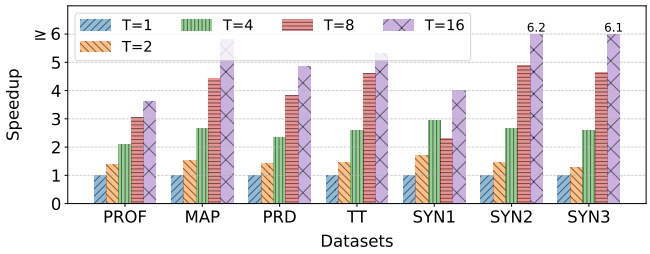


Figure 14. Scalability over Threads (SpecProto-spec).

$> 10^7$)—sequentially merging them takes time. One possible optimization is to pre-merge the parsed fields within each chunk during the parallel decoding.

Table 3 reports the number of bytes re-parsed during the merge phase. For all datasets, the redo cost is negligible. A detailed breakdown of the redo cost indicates that all such costs arise when, during the merge phase, the speculatively parsed field type does not match the required type, leading to a redo. One relatively trivial case is when types such as `bool`, `int32`, or `uint64` fail to match; a possible optimization would be to handle such types explicitly using type casting to avoid redo. Another case occurs when a `string` is mismatched with an `int array`, or when two submessages fail to match, in which case re-parsing is required. The latter accounts for the majority of the redo cost in the SYN3 dataset.

The experiments on our tested datasets demonstrate only limited misspeculation and its potential cost. In the worst case, if there are many fake tags, or if the string or bytes fields overlap substantially with submessage data, the chances of misspeculation can increase significantly.

Benefits of Type Prioritization. Figure 13 compares the execution time with and without type prioritization. The results indicate that this optimization is critical for most datasets. Without the heuristics for candidate type selection, it would take backtracking many more attempts to find the correct type. Furthermore, in the merge phase, validation failures led to expensive redos. A good type prioritization heuristics can help determine a tag's type more quickly and reduce the likelihood of validation failure.

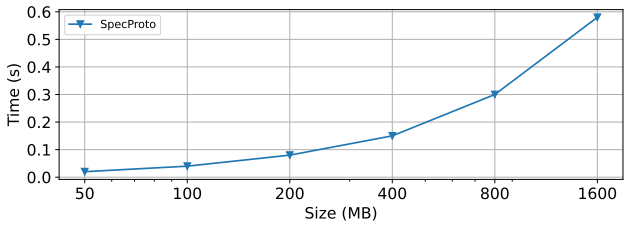


Figure 15. Scalability over Message Size (SpecProto-spec).

6.5 Scalability

Figure 14 shows the speedup of spec with varying numbers of threads. We observe that performance gains from more threads gradually diminish, due to the memory-intensive nature of the parsing process. Separately, when using 8 threads on the SYN1 dataset, one of the chunks experienced a misspeculation that triggered substantial overhead, slowing down the overall execution time.

Figure 15 illustrates how spec scales with increasing binary size using the TT dataset. The results indicate that spec exhibits near-linear scaling.

7 Related Work

In this section, we review prior work on parallel processing of serialized data and discuss how our techniques may extend to other binary serialization formats.

Parallel Processing of Textual Data. Extensive research has explored parallel querying over raw textual data formats such as JSON, XML, and CSV, with most solutions built on formal querying models like NFAs or DFAs. For instance, to enable parallel XPath querying, Ogden et al. [24] proposed parallel pushdown transducers (PPT), which evaluate all possible query states simultaneously when processing an XML partition. Jiang and Zhao [10] improved PPT efficiency by pruning infeasible states using the XML schema.

Prior works on accelerating JSON processing has largely focused on exploiting bit-wise and byte-wise parallelism. For example, Mison [17] builds bitmap index for large JSON records, while Pison [8] improves the indexing scalability by employing multiple cores. simdjson [16] employs bitwise and SIMD operations to validate and tokenize JSON data prior to building a parse tree. JPstream [9] leverages speculative streaming to evaluate a JSONPath query in parallel. Finally, JSONSki [11] utilizes bit-parallelism to fast-forward over irrelevant data during streaming query matching.

While not directly applicable, the "divide and conquer" and state speculation strategies from the above works inspired the approaches proposed in this work.

Acceleration of Protobuf. Existing research has explored hardware-based approaches. Early efforts such as UDP target ETL workloads by accelerating data transformation using

programmable hardware [2]. More recently, Optimus Prime proposes a general-purpose accelerator for serialization and deserialization by exploiting field-level parallelism within message schemas [27]. One study proposes a near-core accelerator that combines specialized execution units with optimized memory and layout representations [13]. Another offloads Protobuf deserialization to SmartNICs, exploiting message-level parallelism across NIC cores and pipelined execution with host computation [1].

In software, Google's μpb provides a high-performance C decoding runtime that uses table-driven parsing and arena-based memory management to reduce per-field dispatch and allocation overhead, but it remains largely single-message, single-core in nature [6].

As far as we are aware, no previous research has explored parallelizing the decoding of a single serialized binary.

Possible Extension to Other Binary Formats. Given the similarities among binary data formats, some of our parallel decoding ideas proposed for Protobuf could be extended to other formats. For instance, Thrift is also schema-based, like Protobuf, with explicit field types and identifiers. Our speculative decoding method can be directly applied to Thrift by identifying the type and field ID, speculatively decoding fields, validating the speculation, and merging the correct parses. Our skimming-based decoding cannot be applied to Thrift, as it does not encode the byte length in the binary data for types such as lists and structs.

For schema-less formats such as MessagePack and BSON, our skimming-based decoding method is applicable, thanks to the encoded length information. However, speculative decoding requires strict schemas, which often times are not available for such types of data formats.

8 Conclusion

This work tackles a fundamental issue in Protobuf decoding—encoding-induced dependencies that limit parallelism. To address this issue, it presents two methods: skimming-based and speculative parallelization. Skimming exploits the length-prefixing design of Protobuf to find independent decoding tasks, while speculative parallelization cuts the binary into even chunks and leverages schema constraints to infer the decoding state for each chunk. Evaluation based on real and synthetic datasets shows that both parallelization methods deliver substantial speedups on multicore CPUs.

Acknowledgments

We thank all reviewers for their constructive feedback and our shepherd, Dr. Babak Falsafi, for his efforts in helping with the paper's revision. This material is based upon work supported by the National Science Foundation under Grants No. 1751392 and 2106383. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the National Science Foundation.

References

[1] Shiyi Cao, Salvatore Di Girolamo, and Torsten Hoefler. 2022. Accelerating Data Serialization/Deserialization Protocols with In-Network Compute. In *2022 IEEE/ACM International Workshop on Exascale MPI (ExaMPI)*. 22–30. doi:10.1109/ExaMPI56604.2022.00008

[2] Yuanwei Fang, Chen Zou, Aaron J. Elmore, and Andrew A. Chien. 2017. UDP: a programmable accelerator for extract-transform-load workloads and more. In *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture* (Cambridge, Massachusetts) *(MICRO-50 '17)*. Association for Computing Machinery, New York, NY, USA, 55–68. doi:10.1145/3123939.3123983

[3] Apache Software Foundation. 2024. Apache Avro. https://avro.apache.org/. Accessed: 2024-10-05.

[4] Milos Gligoric, Darko Marinov, and Sam Kamin. 2011. CoDeSe: fast deserialization via code generation. In *Proceedings of the 2011 International Symposium on Software Testing and Analysis* (Toronto, Ontario, Canada) *(ISSTA '11)*. Association for Computing Machinery, New York, NY, USA, 298–308. doi:10.1145/2001420.2001456

[5] Google. 2019. Google Map Directions API. https://developers.google.com/maps/documentation/directions/start/. Accessed: 2019-07-01.

[6] Google. 2024. A small protobuf implementation written in C. https://github.com/protocolbuffers/protobuf/tree/main/upb. Accessed: 2024-10-05.

[7] Google. 2025. Protocol Buffers. https://github.com/protocolbuffers/protobuf. Accessed: 2025-08-20.

[8] Lin Jiang, Junqiao Qiu, and Zhijia Zhao. 2020. Scalable structural index construction for JSON analytics. *Proc. VLDB Endow.* 14, 4 (Dec. 2020), 694–707. doi:10.14778/3436905.3436926

[9] Lin Jiang, Xiaofan Sun, Umar Farooq, and Zhijia Zhao. 2019. Scalable Processing of Contemporary Semi-Structured Data on Commodity Parallel Processors - A Compilation-based Approach. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems* (Providence, RI, USA) *(ASPLOS '19)*. Association for Computing Machinery, New York, NY, USA, 79–92. doi:10.1145/3297858.3304008

[10] Lin Jiang and Zhijia Zhao. 2017. Grammar-aware Parallelization for Scalable XPath Querying. In *Proceedings of the 22nd ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming* (Austin, Texas, USA) *(PPoPP '17)*. Association for Computing Machinery, New York, NY, USA, 371–383. doi:10.1145/3018743.3018772

[11] Lin Jiang and Zhijia Zhao. 2022. JSONSki: streaming semi-structured data with bit-parallel fast-forwarding. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems* (Lausanne, Switzerland) *(ASPLOS '22)*. Association for Computing Machinery, New York, NY, USA, 200–211. doi:10.1145/3503222.3507719

[12] Svilen Kanev, Juan Pablo Darago, Kim Hazelwood, Parthasarathy Ranganathan, Tipp Moseley, Gu-Yeon Wei, and David Brooks. 2015. Profiling a warehouse-scale computer. *SIGARCH Comput. Archit. News* 43, 3S (June 2015), 158–169. doi:10.1145/2872887.2750392

[13] Sagar Karandikar, Chris Leary, Chris Kennelly, Jerry Zhao, Dinesh Parimi, Borivoje Nikolic, Krste Asanovic, and Parthasarathy Ranganathan. 2021. A Hardware Accelerator for Protocol Buffers. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture* (Virtual Event, Greece) *(MICRO '21)*. Association for Computing Machinery, New York, NY, USA, 462–478. doi:10.1145/3466752.3480051

[14] Martin Kleppmann. 2017. Encoding and Evolution. In *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media, 112–121.

[15] Bruno Krebs. 2017. Beating JSON performance with Protobuf. https://auth0.com/blog/beating-json-performance-with-protobuf/. Accessed: 2024-11-01.
[16] Geoff Langdale and Daniel Lemire. 2019. Parsing gigabytes of JSON per second. *The VLDB Journal* 28, 6 (Oct. 2019), 941–960. doi:10.1007/s00778-019-00578-5
[17] Yinan Li, Nikos R. Katsipoulakis, Badrish Chandramouli, Jonathan Goldstein, and Donald Kossmann. 2017. Mison: a fast JSON parser for data analytics. *Proc. VLDB Endow.* 10, 10 (June 2017), 1118–1129. doi:10.14778/3115404.3115416
[18] Gang Liao, Ye Liu, Yonghua Ding, Le Cai, and Jianjun Chen. 2024. SFVInt: Simple, Fast and Generic Variable-Length Integer Decoding using Bit Manipulation Instructions. In *Proceedings of the 20th International Workshop on Data Management on New Hardware* (Santiago, AA, Chile) *(DaMoN '24)*. Association for Computing Machinery, New York, NY, USA, Article 1, 9 pages. doi:10.1145/3662010.3663439
[19] Wei Lu, Kenneth Chiu, and Yinfei Pan. 2006. A Parallel Approach to XML Parsing. In *Proceedings of the 7th IEEE/ACM International Conference on Grid Computing (GRID '06)*. IEEE Computer Society, USA, 223–230. doi:10.1109/ICGRID.2006.311019
[20] Sergey Melnik, Andrey Gubarev, Jing Jing Long, Geoffrey Romer, Shiva Shivakumar, Matt Tolton, Theo Vassilakis, Hossein Ahmadi, Dan Delorey, Slava Min, Mosha Pasumansky, and Jeff Shute. 2020. Dremel: a decade of interactive SQL analysis at web scale. *Proc. VLDB Endow.* 13, 12 (Aug. 2020), 3461–3472. doi:10.14778/3415478.3415568
[21] MessagePack Developers. 2024. MessagePack. https://msgpack.org/index.html. Accessed: 2024-10-05.
[22] Microsoft. 2024. mimalloc. https://github.com/microsoft/mimalloc. Accessed: 2024-10-10.
[23] MongoDB. 2024. BSON. https://bsonspec.org/. Accessed: 2024-10-05.
[24] Peter Ogden, David Thomas, and Peter Pietzuch. 2013. Scalable XML query processing using parallel pushdown transducers. *Proc. VLDB Endow.* 6, 14 (sep 2013), 1738–1749. doi:10.14778/2556549.2556558
[25] OpenMP Architecture Review Board. 2024. OpenMP. https://www.openmp.org/. Accessed: 2024-10-10.
[26] Pallets. 2024. Jinja. https://github.com/pallets/jinja. Accessed: 2024-11-03.
[27] Arash Pourhabibi, Siddharth Gupta, Hussein Kassir, Mark Sutherland, Zilu Tian, Mario Paulo Drumond, Babak Falsafi, and Christoph Koch. 2020. Optimus Prime: Accelerating Data Transformation in Servers. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems* (Lausanne, Switzerland) *(ASPLOS '20)*. Association for Computing Machinery, New York, NY, USA, 1203–1216. doi:10.1145/3373376.3378501
[28] Daniel Persson Proos and Niklas Carlsson. 2020. Performance Comparison of Messaging Protocols and Serialization Formats for Digital Twins in IoV. *2020 IFIP Networking Conference (Networking)* (2020), 10–18. https://api.semanticscholar.org/CorpusID:220669345
[29] Mark Slee, Aditya Agarwal, and Marc Kwiatkowski. 2007. *Thrift: Scalable Cross-Language Services Implementation.* Technical Report. Facebook. https://thrift.apache.org/static/files/thrift-20070401.pdf
[30] The Go Authors. 2024. runtime/pprof. https://pkg.go.dev/runtime/pprof. Accessed: 2024-09-19.
[31] TwitchTV. 2019. Serialization and deserialization of large messages can be slow #166. https://github.com/twitchtv/twirp/issues/166. Accessed: 2024-11-06.
[32] Twitter. 2019. Twitter Developer API. https://developer.twitter.com/en/docs/. Accessed: 2019-07-01.
[33] Walmart. 2019. Product Lookup API. https://developer.walmartlabs.com/docs. Accessed: 2019-05-10.
[34] Zhijia Zhao, Bo Wu, and Xipeng Shen. 2014. Challenging the "embarrassingly sequential": parallelizing finite state machine-based computations through principled speculation. In *Proceedings of the 19th International Conference on Architectural Support for Programming Languages and Operating Systems* (Salt Lake City, Utah, USA) *(ASPLOS '14)*. Association for Computing Machinery, New York, NY, USA, 543–558. doi:10.1145/2541940.2541989

A Artifact Appendix

A.1 Abstract

The source code and evaluation scripts for SpecProto are available at https://github.com/AutomataLab/SpecProto. The artifact provides a collection of scripts to reproduce the results and figures presented in the paper, including execution time (Figure 11), memory overhead (Table 2), speculation cost breakdown (Table 3), time breakdown (Figure 12), benefits of type prioritization (Figure 13), scalability over threads (Figure 14), and scalability over message size (Figure 15). The artifact also provides links to the datasets. It has been tested on Linux systems with 16 physical CPU cores and at least 16 GB of memory. The results produced by the artifact should be similar to those reported in the paper.

A.2 Artifact check-list (meta-information)

- **Algorithm:** Parallel Protobuf Decoding
- **Program:** SpecProto
- **Compilation:** G++ 9.2.1
- **Data set:** Links to seven publicly available Protobuf datasets
- **Run-time environment:** Developed and tested on Rocky Linux 8.
- **Hardware:** All experiments ran on a server node with two Intel Xeon E5-2683 v4 processors and 512GB RAM.
- **Execution:** Command-line execution of compiled binaries with specified input files and parameters
- **Metrics:** Execution time, memory overhead, processed bytes
- **Output:** Logs reporting execution time and memory overhead
- **Experiments:** Full reproduction of Figures 11–15 and Tables 2-3 (execution time, memory overhead, cost and time breakdown, scalability)
- **How much disk space is required (approximately)?:** 10GB
- **How much time is needed to prepare workflow (approximately)?:** 10 minutes (cloning repository, installing dependencies)
- **How much time is needed to complete experiments (approximately)?:** 1-3 hours
- **Publicly available?:** Yes
- **Code licenses (if publicly available)?:** MIT License

A.3 Description

A.3.1 How to access. The full artifact, including source code, scripts, and documentation, is available on GitHub: https://github.com/AutomataLab/SpecProto. Users can clone the repository using Git or download it as a ZIP file.

A.3.2 Hardware dependencies. To reproduce the results reported in the paper, we recommend using a machine with 16 physical CPU cores and at least 16 GB of memory.

A.3.3 Software dependencies. We recommend running the artifact on Rocky Linux 8, though similar Linux systems should also work. We provide a script to install the necessary dependencies (e.g., Protobuf).

A.3.4 Datasets. We include links to the necessary datasets in our artifact.

A.4 Installation

The artifact has been tested with G++ 9.2.1 and CMake 3.26.4. Other compatible versions may also work. Compilation and execution scripts are provided in the repository. Detailed instructions can be found in `README.md`.

A.5 Experiment workflow

Clone the repository and move it to the desired directory. Generate and build the code. Execute the compiled binaries with the available datasets. `README.md` provides step-by-step instructions for running the experiments.

A.6 Evaluation and expected results

We provide the necessary commands to generate the figures and tables presented in the evaluation section. The results should be similar to those reported in our paper.

A.7 Experiment customization

Users can add new datasets and schemas and place them in `dataset/` and `schema/`, respectively. Then run the relevant commands (see the examples in `README.md`).